AI-Powered Prototype Development

TariffWise Footwear Classification Prototype

Musse Abayneh | MPIM-6750-140 | Module 5 Hands-On

Overview

TariffWise is an AI tariff classification tool for small and medium importers. The prototype implements one path from the Module 4 design, footwear classification. It takes a product, runs an interview that captures the facts that change the code, suggests a Harmonized Tariff Schedule heading, and stores a documented audit record. US importers are responsible for the code they declare, even when a broker files the entry. The tool is built to reach a defensible code and document the reasoning behind it.

Tool selection

Claude was selected to generate the TariffWise mockup and write the prototype code. The task is reasoning heavy classification. It must produce a defensible Harmonized Tariff Schedule code and a documented rationale for each decision. This needs multi step reasoning, close adherence to detailed constraints, and plain language explanation. Claude keeps a long specification in view while building. Through vibe coding it generated the Python prototype as a Jupyter notebook that runs in a Conda environment, and Claude Code can run that notebook locally and step through the logic. The alternatives were set aside on fit

rather than capability. ChatGPT runs notebooks natively, which is convenient. Gemini integrates well with Colab, which helps with hosting more than with reaching a defensible code. Cursor is built for editing large codebases rather than a single notebook. All four could build a prototype of this kind.

Coding process

The prototype was built through vibe coding. The build was steered in natural language rather than typed line by line. It moved in stages, and each stage fed the next.

The mockup came first. A high fidelity visual of the five screens fixed the target the notebook had to hit. The code then had a concrete specification to match, including the worked example, the captured facts, the rationale, and the audit record.

The notebook was built section by section. The classification logic is a small ordered rule set over HTS Chapter 64, where the heading turns on the outer sole and the upper material. The interview captures only the facts that change the code. The rationale and the audit record are generated from those same facts, so the explanation stays tied to the answers.

Two steps corrected the build. The first run classified the canvas sneaker as a routine suggestion, but the design routed it to expert review. The rule set was revised to treat a borderline material split as a trigger for human sign off. The

second change separated the suggested stage from the saved stage, so a code moves from suggested, to accepted, to saved in the audit trail.

The notebook was run from start to finish after each change, and the outputs were checked against the mockup. The history table matched the design. The worked example reached the expected heading with a rationale tied to the interview.

Results

The prototype runs the worked example without input. A women's running shoe with a textile upper, a rubber sole, athletic use, no ankle coverage, and no metal toe cap reaches heading 6404.11.9000 with a four step rationale. The engine handles other cases. A leather ankle boot reaches 6403.99.6075. A steel toe work shoe returns no code and routes to expert review, because the metal toe cap changes the analysis. The rule set is a small illustrative subset of Chapter 64, and the codes are plausible but not legally verified. This matches the product, which suggests and explains rather than files the entry.

Appendix

A. AI prompts used

The build was conversational. These are the representative prompts that traced the work, from grounding through correction and testing.

1. Using my Module 2 TariffWise market research and my Module 4 five screen design, build a Module 5 prototype of the footwear classification path.
2. Generate a high fidelity HTML mockup of the five screens. Keep it document grade, make the HTS code the hero of the result screen, and include an exportable audit record.
3. Build a Jupyter notebook that runs the footwear interview, applies a rule set over HTS Chapter 64, suggests a code, generates a numbered rationale, and prints an audit record. It should run top to bottom with no typing, and add an optional interactive cell.
4. The canvas sneaker should route to expert review the way the design shows. Model a borderline material split as a trigger for review.
5. Separate the suggested stage from the saved stage so the audit record reads Saved after the user accepts it.
6. Classify a leather ankle boot and a steel toe work shoe so I can see the expert review path.

B. Prototype draft, screenshots

Screen flow mockup (five screens).

TARIFFWISE · PROTOTYPE MOCKUP

Footwear classification, end to end

An AI tariff-classification tool for small and mid-size importers. The flow moves from rough product input, through an AI interview that pins down the facts that change the code, to a documented result and a stored audit record.

Worked example Women's running shoe **Niche** Footwear (HTS Chapter 64) **Fidelity** High-fidelity, five screens

---

01 / 05 **Product input**

The user enters what they know. The AI fills the gaps in the next step.

TariffWise — New classification | History | Settings

Classify a new product

Enter product details. The AI asks follow-up questions before suggesting a code.

Product name

Women's running shoe

Product category

Footwear

Material description

Textile upper, rubber outsole, metal eyelets

Upload product spec or image

Drop a spec sheet or product photo, or browse

Start classification

02 / 05 **AI classification interview**

The AI asks only the questions that change the code. The working code updates live.

TariffWise — New classification | History | Settings

AI classification interview

A short interview that mirrors how a customs expert finds a product's essential character.

What material covers the largest surface area of the upper?

Textile, about 70 percent

Does the shoe cover the ankle?

No, it sits below the ankle

Is there a protective metal toe cap?

Type your answer **Send**

WORKING CLASSIFICATION

Upper material	**Textile (majority)**
Ankle coverage	**No**
Metal toe cap	**No**

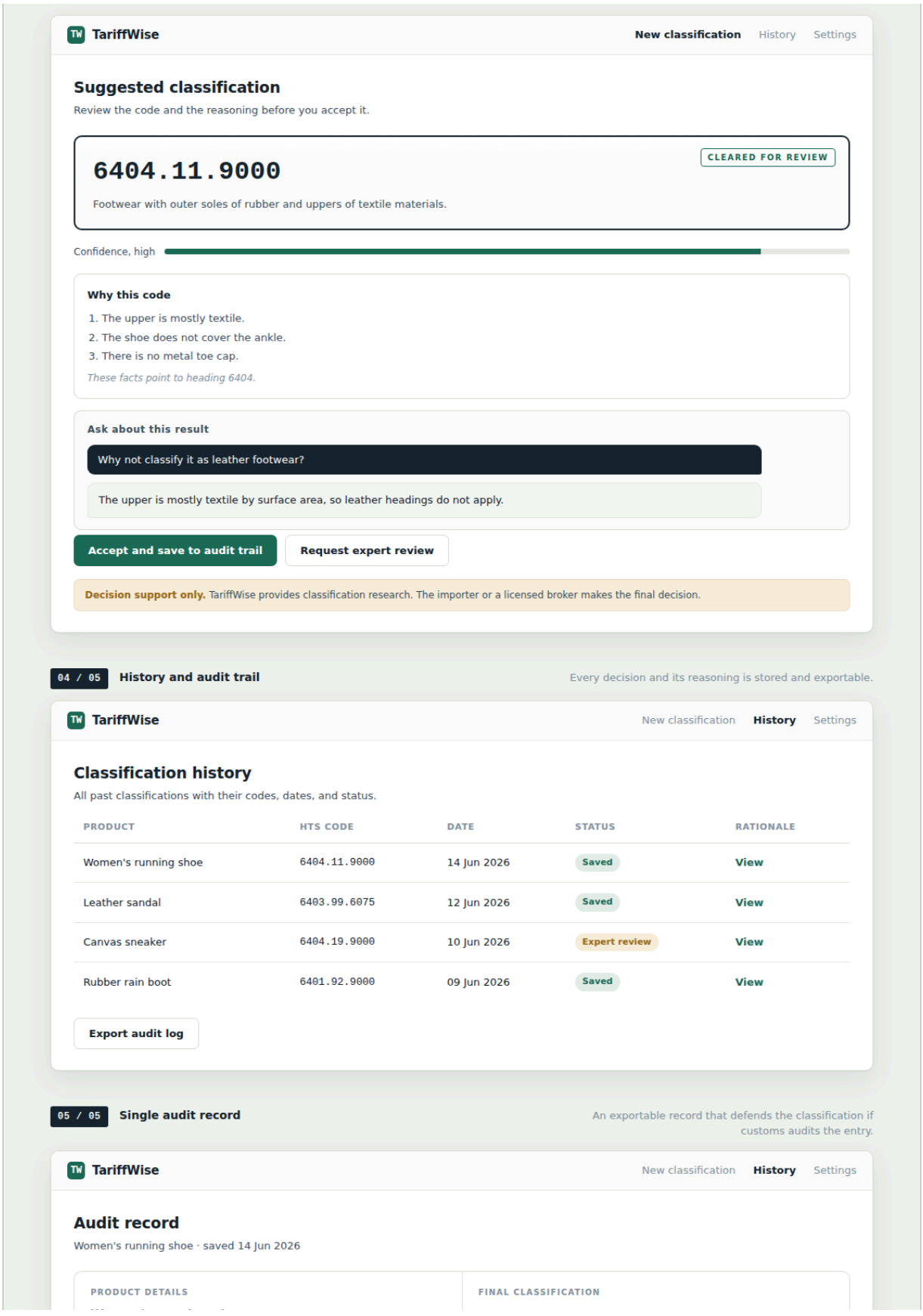
SUGGESTED HEADING

6404

Footwear, textile upper

03 / 05 **Result and rationale**

A documented code, a numbered rationale, and a chat to question it.


TariffWise
New classification
History
Settings
Suggested classification
Review the code and the reasoning before you accept it.
6404.11.9000
CLEARED FOR REVIEW
Footwear with outer soles of rubber and uppers of textile materials.
Confidence, high
Why this code
1. The upper is mostly textile.
2. The shoe does not cover the ankle.
3. There is no metal toe cap.
These facts point to heading 6404.
Ask about this result
Why not classify it as leather footwear?
The upper is mostly textile by surface area, so leather headings do not apply.
Accept and save to audit trail
Request expert review
Decision support only. TariffWise provides classification research. The importer or a licensed broker makes the final decision.
04 / 05
History and audit trail
Every decision and its reasoning is stored and exportable.
TariffWise
New classification
History
Settings
Classification history
All past classifications with their codes, dates, and status.
PRODUCT
HTS CODE
DATE
STATUS
RATIONALE
Women's running shoe
6404.11.9000
14 Jun 2026
Saved
View
Leather sandal
6403.99.6075
12 Jun 2026
Saved
View
Canvas sneaker
6404.19.9000
10 Jun 2026
Expert review
View
Rubber rain boot
6401.92.9000
09 Jun 2026
Saved
View
Export audit log
05 / 05
Single audit record
An exportable record that defends the classification if customs audits the entry.
TariffWise
New classification
History
Settings
Audit record
Women's running shoe · saved 14 Jun 2026
PRODUCT DETAILS
FINAL CLASSIFICATION

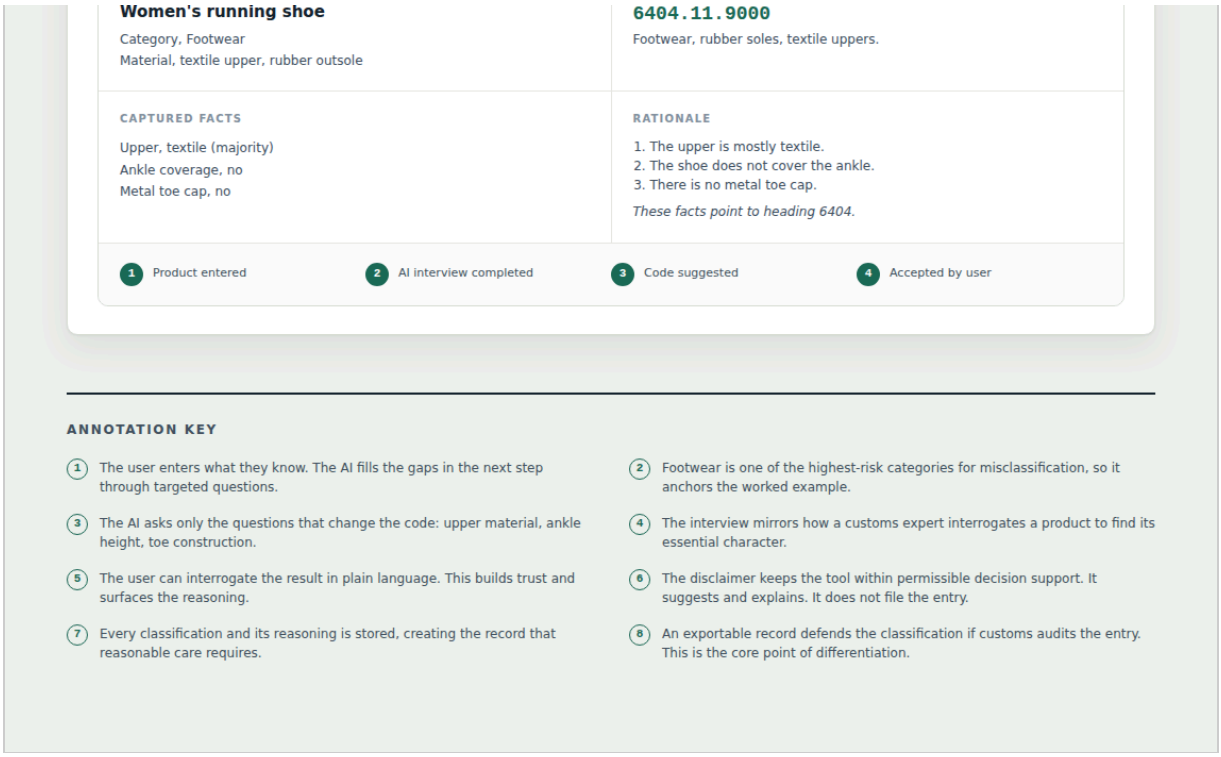


Figure 1. The five screen TariffWise flow, from product input to the stored audit record.

Notebook running, worked example.

In [4]:

```
shoe_facts = {
    "upper_material": "textile",       # textile, about 70 percent of the surface
    "outsole_material": "rubber_plastic",
    "athletic": True,                  # a running shoe
    "covers_ankle": False,             # sits below the ankle
    "metal_toe_cap": False,
    "waterproof": False,
}

shoe_interview = [
    ("What material covers the largest surface area of the upper?",
     "Textile, about 70 percent", "upper_material", "textile"),
    ("What is the outer sole made of?",
     "Rubber", "outsole_material", "rubber_plastic"),
    ("Is this athletic or sports footwear?",
     "Yes, a running shoe", "athletic", True),
    ("Does the shoe cover the ankle?",
     "No, it sits below the ankle", "covers_ankle", False),
    ("Is there a protective metal toe cap?",
     "No", "metal_toe_cap", False),
]

run_interview("Women's running shoe", shoe_interview, shoe_facts)
result = classify(shoe_facts)

print("\n" + "=" * 64)
print("SUGGESTED CLASSIFICATION")
print("=" * 64)
print(f"  HTS code:     {result['code']}")
print(f"  Description:  {result['description']}")
print(f"  Confidence:   {result['confidence']}")
print(f"  Status:       {result['status']}")
print("\n  Why this code:")
for i, line in enumerate(result["rationale"], 1):
    print(f"    {i}. {line}")
```

```
================================================================
AI CLASSIFICATION INTERVIEW — Women's running shoe
================================================================

  TariffWise:  What material covers the largest surface area of the upper?
  Importer:    Textile, about 70 percent
     -> working heading: 6404 (likely, sole pending)

  TariffWise:  What is the outer sole made of?
  Importer:    Rubber
     -> working heading: 6404 (textile upper)

  TariffWise:  Is this athletic or sports footwear?
  Importer:    Yes, a running shoe
     -> working heading: 6404 (textile upper)

  TariffWise:  Does the shoe cover the ankle?
  Importer:    No, it sits below the ankle
     -> working heading: 6404 (textile upper)

  TariffWise:  Is there a protective metal toe cap?
  Importer:    No
     -> working heading: 6404 (textile upper)

----------------------------------------------------------------
Interview complete. All code-relevant facts captured.

================================================================
SUGGESTED CLASSIFICATION
================================================================
  HTS code:     6404.11.9000
  Description:  Footwear, rubber/plastic soles, textile uppers — athletic.
  Confidence:   high
  Status:       Suggested

  Why this code:
    1. The upper is mostly textile by surface area.
    2. The outer sole is rubber or plastic, which points to heading 6404.
    3. The shoe does not cover the ankle.
    4. It is athletic footwear, which falls under subheading 6404.11.
```

Figure 2. The notebook runs the interview and reaches heading 6404.11.9000 with its rationale.

C. Code file, printout

Source from TariffWise_Prototype.ipynb. The notebook file is submitted alongside this PDF.

```
# ------------------------------------------------------------

from dataclasses import dataclass, field
from datetime import date
import pandas as pd

print("TariffWise prototype ready.")

# Each rule is: a test over the captured facts -> (heading, subheading, description).
# Ordered most specific first. This mirrors how a customs reviewer narrows the essential
character.

HEADINGS = {
    "6401.92.9000": "Waterproof footwear, rubber/plastic soles and uppers, covering
ankle.",
    "6402.99.3165": "Other footwear, rubber/plastic soles and uppers.",
    "6403.99.6075": "Footwear, rubber/plastic/leather soles, leather uppers.",
    "6404.11.9000": "Footwear, rubber/plastic soles, textile uppers — athletic.",
    "6404.19.9000": "Footwear, rubber/plastic soles, textile uppers — other.",
    "6405.90.9060": "Other footwear, not elsewhere specified.",
}

def classify(facts):
    """Suggest a code, then apply a borderline check that can route to expert review."""
    result = _classify_core(facts)
    # A borderline material split keeps the leading code but asks a human to sign off.
    if facts.get("borderline") and result["code"] is not None:
        result["confidence"] = "medium"
        result["status"] = "Expert review"
        result["rationale"].append(
            "The upper material split is borderline, so the case is routed for expert
review.")
    return result

def _classify_core(facts):
    """Take captured facts, return a code, description, rationale, confidence, and
status."""
    upper   = facts["upper_material"]      # 'textile' | 'leather' | 'rubber_plastic'
    sole    = facts["outsole_material"]    # 'rubber_plastic' | 'leather'
    athletic = facts["athletic"]           # bool
    ankle    = facts["covers_ankle"]       # bool
    toecap   = facts["metal_toe_cap"]      # bool
    waterproof = facts.get("waterproof", False)

    rationale = []
    status = "Suggested"
    confidence = "high"

    # A protective metal toe cap shifts the analysis. Be honest and route to a human.
    if toecap:
        rationale.append("A protective metal toe cap is present, which changes the
subheading analysis.")
        rationale.append("This case is routed for expert review rather than auto-
suggested.")
        return {"code": None, "description": "Requires expert review",
                "rationale": rationale, "confidence": "low", "status": "Expert review"}

    # Waterproof rubber/plastic one-piece footwear -> 6401.
    if waterproof and upper == "rubber_plastic" and sole == "rubber_plastic":
        rationale.append("The upper and sole are rubber or plastic and the footwear is
waterproof.")
```

```
        rationale.append("It is not assembled by stitching, which points to heading 6401.")
        if ankle:
            rationale.append("It covers the ankle, consistent with subheading 6401.92.")
        return {"code": "6401.92.9000", "description": HEADINGS["6401.92.9000"],
                "rationale": rationale, "confidence": confidence, "status": status}

    # Textile uppers with rubber/plastic soles -> 6404.
    if upper == "textile" and sole == "rubber_plastic":
        rationale.append("The upper is mostly textile by surface area.")
        rationale.append("The outer sole is rubber or plastic, which points to heading
6404.")
        rationale.append("The shoe does not cover the ankle." if not ankle
                         else "The shoe covers the ankle.")
        if athletic:
            rationale.append("It is athletic footwear, which falls under subheading
6404.11.")
            code = "6404.11.9000"
        else:
            rationale.append("It is not athletic footwear, which falls under subheading
6404.19.")
            code = "6404.19.9000"
        return {"code": code, "description": HEADINGS[code],
                "rationale": rationale, "confidence": confidence, "status": status}

    # Leather uppers -> 6403.
    if upper == "leather" and sole in ("rubber_plastic", "leather"):
        rationale.append("The upper is mostly leather by surface area.")
        rationale.append("The outer sole is rubber, plastic, or leather, which points to
heading 6403.")
        return {"code": "6403.99.6075", "description": HEADINGS["6403.99.6075"],
                "rationale": rationale, "confidence": "medium", "status": status}

    # Rubber/plastic uppers, not waterproof -> 6402.
    if upper == "rubber_plastic" and sole == "rubber_plastic":
        rationale.append("The upper and sole are rubber or plastic and the footwear is not
waterproof.")
        rationale.append("This points to heading 6402.")
        return {"code": "6402.99.3165", "description": HEADINGS["6402.99.3165"],
                "rationale": rationale, "confidence": "medium", "status": status}

    # Fallback.
    rationale.append("The captured facts do not match a single heading cleanly.")
    rationale.append("The case is routed for expert review.")
    return {"code": "6405.90.9060", "description": HEADINGS["6405.90.9060"],
            "rationale": rationale, "confidence": "low", "status": "Expert review"}

print("Loaded", len(HEADINGS), "candidate headings and the classification logic.")

def run_interview(product_name, qa_pairs, facts):
    """Print the interview as a transcript and show the working heading update live."""
    print("=" * 64)
    print(f"AI CLASSIFICATION INTERVIEW — {product_name}")
    print("=" * 64)
    captured = {}
    for question, answer, fact_key, fact_value in qa_pairs:
        print(f"\n  TariffWise:  {question}")
        print(f"  Importer:    {answer}")
        captured[fact_key] = fact_value
        # Recompute a rough working heading from what we know so far.
        working = working_heading(captured)
        print(f"     -> working heading: {working}")
    print("\n" + "-" * 64)
    print("Interview complete. All code-relevant facts captured.")
    return facts
```

```
def working_heading(captured):
    """A light, partial guess used only to animate the live panel during the interview."""
    if captured.get("upper_material") == "textile" and captured.get("outsole_material") ==
"rubber_plastic":
        return "6404 (textile upper)"
    if captured.get("upper_material") == "textile":
        return "6404 (likely, sole pending)"
    if captured.get("upper_material") == "leather":
        return "6403 (leather upper)"
    if captured.get("upper_material") == "rubber_plastic":
        return "6401/6402 (rubber upper)"
    return "pending"

shoe_facts = {
    "upper_material": "textile",       # textile, about 70 percent of the surface
    "outsole_material": "rubber_plastic",
    "athletic": True,                  # a running shoe
    "covers_ankle": False,             # sits below the ankle
    "metal_toe_cap": False,
    "waterproof": False,
}

shoe_interview = [
    ("What material covers the largest surface area of the upper?",
     "Textile, about 70 percent", "upper_material", "textile"),
    ("What is the outer sole made of?",
     "Rubber", "outsole_material", "rubber_plastic"),
    ("Is this athletic or sports footwear?",
     "Yes, a running shoe", "athletic", True),
    ("Does the shoe cover the ankle?",
     "No, it sits below the ankle", "covers_ankle", False),
    ("Is there a protective metal toe cap?",
     "No", "metal_toe_cap", False),
]

run_interview("Women's running shoe", shoe_interview, shoe_facts)
result = classify(shoe_facts)

print("\n" + "=" * 64)
print("SUGGESTED CLASSIFICATION")
print("=" * 64)
print(f"  HTS code:     {result['code']}")
print(f"  Description:  {result['description']}")
print(f"  Confidence:   {result['confidence']}")
print(f"  Status:       {result['status']}")
print("\n  Why this code:")
for i, line in enumerate(result["rationale"], 1):
    print(f"    {i}. {line}")

FOLLOW_UPS = {
    "leather": ("Why not classify it as leather footwear?",
                "The upper is mostly textile by surface area, so the leather heading 6403
does not apply."),
    "waterproof": ("Could this be waterproof footwear under 6401?",
                   "Heading 6401 needs rubber or plastic uppers assembled without
stitching. This upper is textile, so 6401 does not apply."),
}

def ask_about(topic):
    q, a = FOLLOW_UPS.get(topic, ("", "No canned answer for that question yet."))
    if q:
        print(f"  Importer:    {q}")
    print(f"  TariffWise:  {a}")
```

```
ask_about("leather")

# The importer reviews the suggestion on Screen 3 and clicks "Accept and save to audit
trail".
accepted = dict(result)
accepted["status"] = "Saved"

def audit_record(product_name, facts, result, record_date=None):
    record_date = record_date or date.today()
    fact_labels = {
        "upper_material":   ("Upper material",  {"textile":"Textile
(majority)","leather":"Leather (majority)","rubber_plastic":"Rubber or plastic"}),
        "outsole_material": ("Outer sole",      {"rubber_plastic":"Rubber or
plastic","leather":"Leather"}),
        "athletic":         ("Athletic use",    {True:"Yes",False:"No"}),
        "covers_ankle":     ("Ankle coverage",  {True:"Yes",False:"No"}),
        "metal_toe_cap":    ("Metal toe cap",   {True:"Yes",False:"No"}),
    }
    line = "-" * 64
    print(line)
    print(f"AUDIT RECORD — {product_name}".ljust(50) + str(record_date))
    print(line)
    print("PRODUCT DETAILS")
    print(f"  Name:      {product_name}")
    print(f"  Category:  Footwear")
    print("\nCAPTURED FACTS")
    for key,(label,mapping) in fact_labels.items():
        if key in facts:
            print(f"  {label+':':18}{mapping.get(facts[key], facts[key])}")
    print("\nFINAL CLASSIFICATION")
    print(f"  HTS code:    {result['code']}")
    print(f"  Description: {result['description']}")
    print(f"  Status:      {result['status']}")
    print("\nRATIONALE")
    for i,r in enumerate(result["rationale"],1):
        print(f"  {i}. {r}")
    print("\nSTEPS TAKEN")
    print("  1 Product entered  ->  2 AI interview completed  ->  3 Code suggested  ->  4
Accepted by user")
    print(line)

audit_record("Women's running shoe", shoe_facts, accepted, date(2026,6,14))

samples = [
    ("Women's running shoe", date(2026,6,14),
     dict(upper_material="textile", outsole_material="rubber_plastic", athletic=True,
covers_ankle=False, metal_toe_cap=False, waterproof=False)),
    ("Leather sandal", date(2026,6,12),
     dict(upper_material="leather", outsole_material="rubber_plastic", athletic=False,
covers_ankle=False, metal_toe_cap=False, waterproof=False)),
    ("Canvas sneaker", date(2026,6,10),
     dict(upper_material="textile", outsole_material="rubber_plastic", athletic=False,
covers_ankle=False, metal_toe_cap=False, waterproof=False, borderline=True)),
    ("Rubber rain boot", date(2026,6,9),
     dict(upper_material="rubber_plastic", outsole_material="rubber_plastic",
athletic=False, covers_ankle=True, metal_toe_cap=False, waterproof=True)),
]

rows = []
for name, d, facts in samples:
    r = classify(facts)
    # Accepted suggestions are saved to the trail; routed cases keep the review flag.
    shown_status = "Saved" if r["status"] == "Suggested" else r["status"]
```

```
    rows.append({"Product": name, "HTS code": r["code"], "Date": d.strftime("%d %b %Y"),
"Status": shown_status})

history = pd.DataFrame(rows)
history

def interactive_classify():
    print("Describe the footwear. Press Enter to accept the default in brackets.\n")
    def ask(q, options, default):
        raw = input(f"{q} {options} [{default}]: ").strip().lower()
        return raw if raw else default
    facts = {
        "upper_material":   ask("Upper material?", "(textile/leather/rubber_plastic)",
"textile"),
        "outsole_material": ask("Outer sole material?", "(rubber_plastic/leather)",
"rubber_plastic"),
        "athletic":         ask("Athletic footwear?", "(yes/no)", "no") in
("yes","y","true"),
        "covers_ankle":     ask("Covers the ankle?", "(yes/no)", "no") in
("yes","y","true"),
        "metal_toe_cap":    ask("Protective metal toe cap?", "(yes/no)", "no") in
("yes","y","true"),
        "waterproof":       ask("Waterproof?", "(yes/no)", "no") in ("yes","y","true"),
    }
    r = classify(facts)
    print("\nSuggested code:", r["code"], "—", r["description"])
    print("Status:", r["status"], "| Confidence:", r["confidence"])
    print("Rationale:")
    for i,line in enumerate(r["rationale"],1):
        print(f"  {i}. {line}")

# Uncomment the next line to run the interactive interview:
# interactive_classify()
```